

CSE 493S Empirical Methods Assignment

Aadyant Maity and Rahul Bonthu

May 1, 2026

Part 0: implementation

Part 0 is basically the codebase the rest of the assignment builds on and everything lives under `deliverables/part_0/`: the usual scripts (`train.py`, `model.py`, `inference.py`, `tokenizer_utils.py`, `arith_data.py`) plus `part_0_1_contract.py` for the autograder-facing API we sanity-check with the quick configs under `deliverables/part_0/configs/`, for example:

```
python train.py --config configs/sanity_quick.yaml
python train.py --config configs/sanity_suffix_quick.yaml
```

(run from the `EmpiricalHW/` root, or point paths appropriately). Nothing fancy to report here besides “it trains and the contract loads checkpoints”; the interesting experiments start in Part 1.

Part 1.1: modular arithmetic datasets

For Part 1.1 we generated text files of equations $a \text{ op } b = c$ with arithmetic mod p . Addition and subtraction iterate all pairs (a, b) with $0 \leq a, b < p$. Division skips $b = 0$ and uses $b \in \{1, \dots, p-1\}$, so division sets are a bit smaller than add/sub for the same p . We shuffled once with seed 0 then split each file **80% / 10% / 10%** train, validation, and test in that order. Every `meta.json` matches the lines in the matching `.txt` files.

Split sizes

These counts match what is on disk under `deliverables/part_1/1.1/data/`.

Table 1: Number of lines per split for each operator and modulus.

p	op	train	val	test
97	+ / -	7527	940	942
97	/	7449	931	932
113	+ / -	10215	1276	1278
113	/	10124	1265	1267

To regenerate from the course workflow:

```
python arith_data.py --seed 0
```

Part 1.2: addition mod 97 (one-layer, deliverable run)

Part 1.2 is the addition-mod-97 setup with a one-layer model. Our submitted bundle lives under `deliverables/part_1/1.2/checkpoints/p97_add_1L_restart1_seed101/`, with final weights `ckpt_100000.pt` (`max_steps = 100000`, `save_every = 10000`), plus `tokenizer.json`, `metrics.csv`, and resolved config metadata.

Training curves (seed 101)

Figure 1 shows train/val/test loss, token accuracy, and equation accuracy for steps 0–100000 only (we pass `--max-step 100000` to `plot_metrics.py` so the figure matches the deliverable budget exactly).

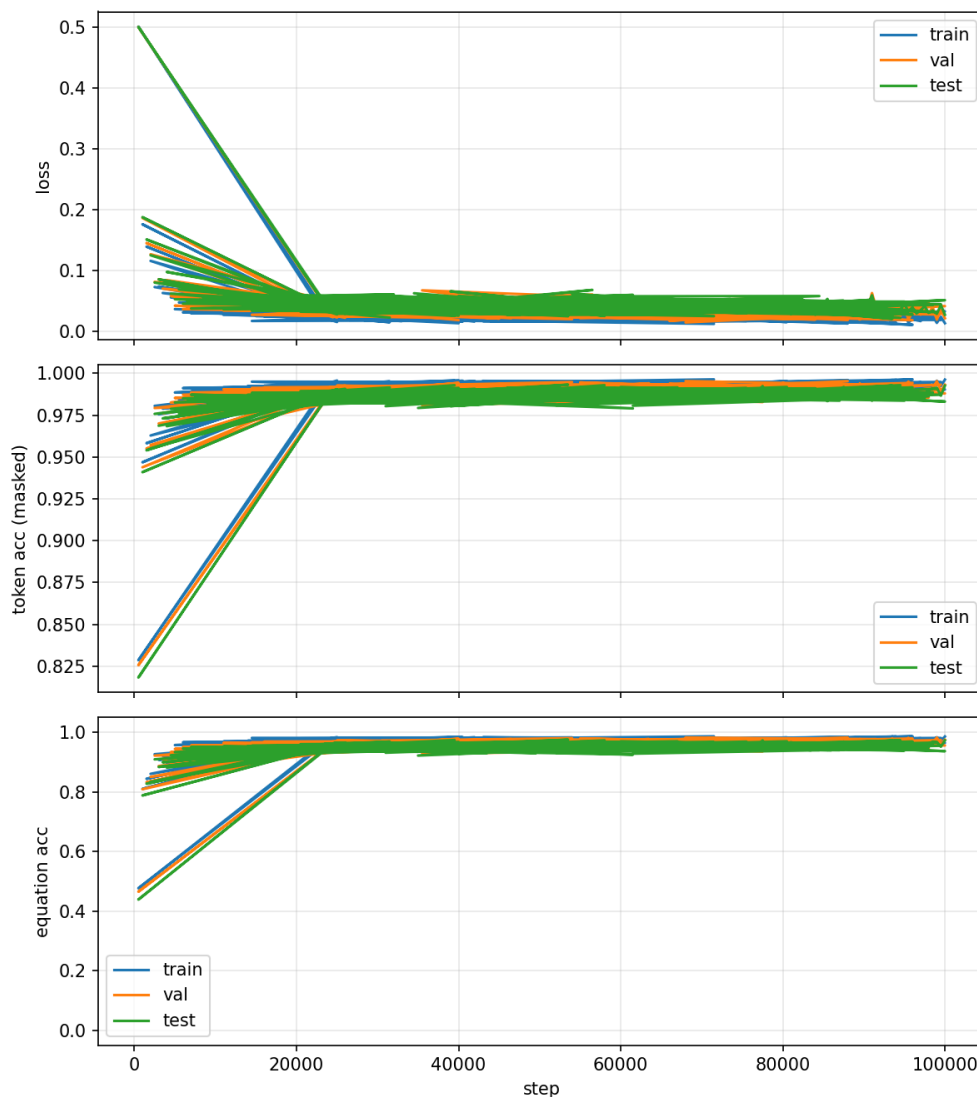


Figure 1: Part 1.2 deliverable run: $p = 97$, addition, one layer, seed 101, 100k steps.

Numbers at the last logged step (seed 101)

At step 100000 (last row of the deliverable `metrics.csv`), we see roughly:

- Train loss ≈ 0.0136 , train token acc ≈ 0.996 .
- Val loss ≈ 0.0218 , val token acc ≈ 0.993 .
- Test loss ≈ 0.0275 , test token acc ≈ 0.993 .
- Train / val / test equation acc ≈ 0.985 , 0.974 , and 0.972 .

Part 1.3: division mod 97 (grokking-style run)

Part 1.3 trains on division modulo 97. The deliverable bundle is under `deliverables/part_1/1.3/checkpoints/part1_3_div_p97/`, with final weights `ckpt_300000.pt`, `tokenizer.json`, `metrics.csv`, and resolved config metadata.

Training curves

Figure 2 shows loss and accuracy vs. step from `metrics.csv`. We generate the PNG with `plot_metrics.py` from that run's CSV (repo-root script, `--max-step` optional).

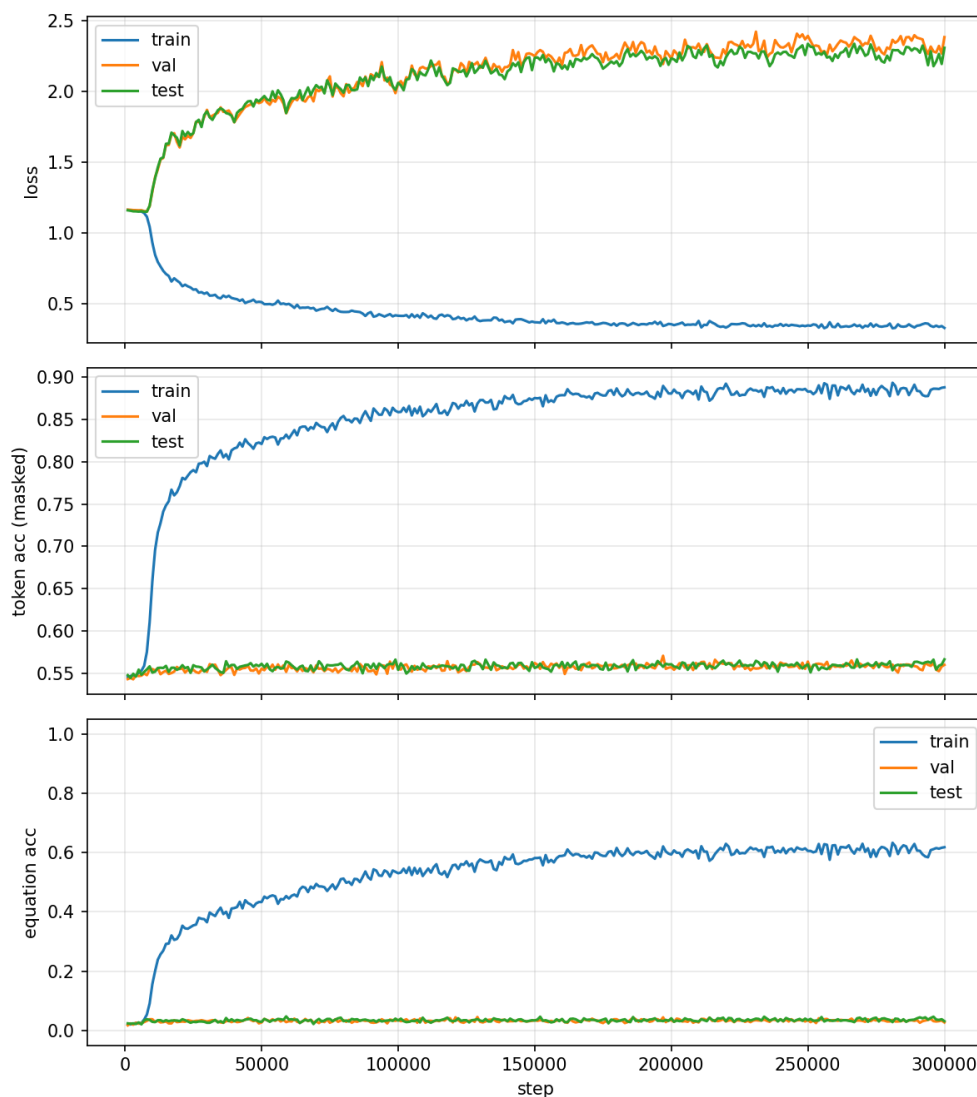


Figure 2: Training / validation / test curves for the division mod 97 experiment (`part1_3_div_p97`).

Numbers at the last logged step

At step 300000 (last row of `metrics.csv`), the CSV reports roughly:

- Train loss ≈ 0.331 , train token acc ≈ 0.888 .
- Val loss ≈ 2.38 , val token acc ≈ 0.560 .
- Test loss ≈ 2.31 , test token acc ≈ 0.566 .
- Train equation acc ≈ 0.618 ; val/test equation acc stayed low on the last rows.

We are not over-interpreting these here; the figure is the main takeaway for us until we write up discussion elsewhere.

How to run inference

From `deliverables/part_1/1.3/code/`, we can use either free-form generation or `predict_answer` on integers. Example:

```
python inference.py --checkpoint_dir ../checkpoints/part1_3_div_p97 \  
    --prompt "3 / 5 = "
```

Part 1.4: ablations (division mod 97)

Part 1.4 compares two tweaks to the Part 1.3-style division setup ($p = 97$, one layer, 300k steps, seed 42). Each ablation has its own checkpoint folder under `deliverables/part_1/1.4/checkpoints/`.

Dropout (dropout = 0.2, weight_decay = 1.0)

Bundle: `part_1/1.4/checkpoints/part1_4_ablation_dropout/`.

Figure 3: `plot_metrics.py` with `--max-step 300000`.



Figure 3: Part 1.4 dropout ablation: division mod 97, dropout = 0.2, 300k steps.

At step 300000 (last row of that run's `metrics.csv`):

- Train loss ≈ 1.16 , train token acc ≈ 0.546 .
- Val loss ≈ 1.16 , val token acc ≈ 0.540 .

- Test loss ≈ 1.16 , test token acc ≈ 0.548 .
- Train / val / test equation acc ≈ 0.021 , 0.016 , and 0.021 .

Low weight decay (`weight_decay = 0.1`, `dropout = 0.0`)

Bundle: `part_1/1.4/checkpoints/part1_4_ablation_wd_low/`.

Figure 4: same `plot_metrics.py` setup with that run's `metrics.csv`.

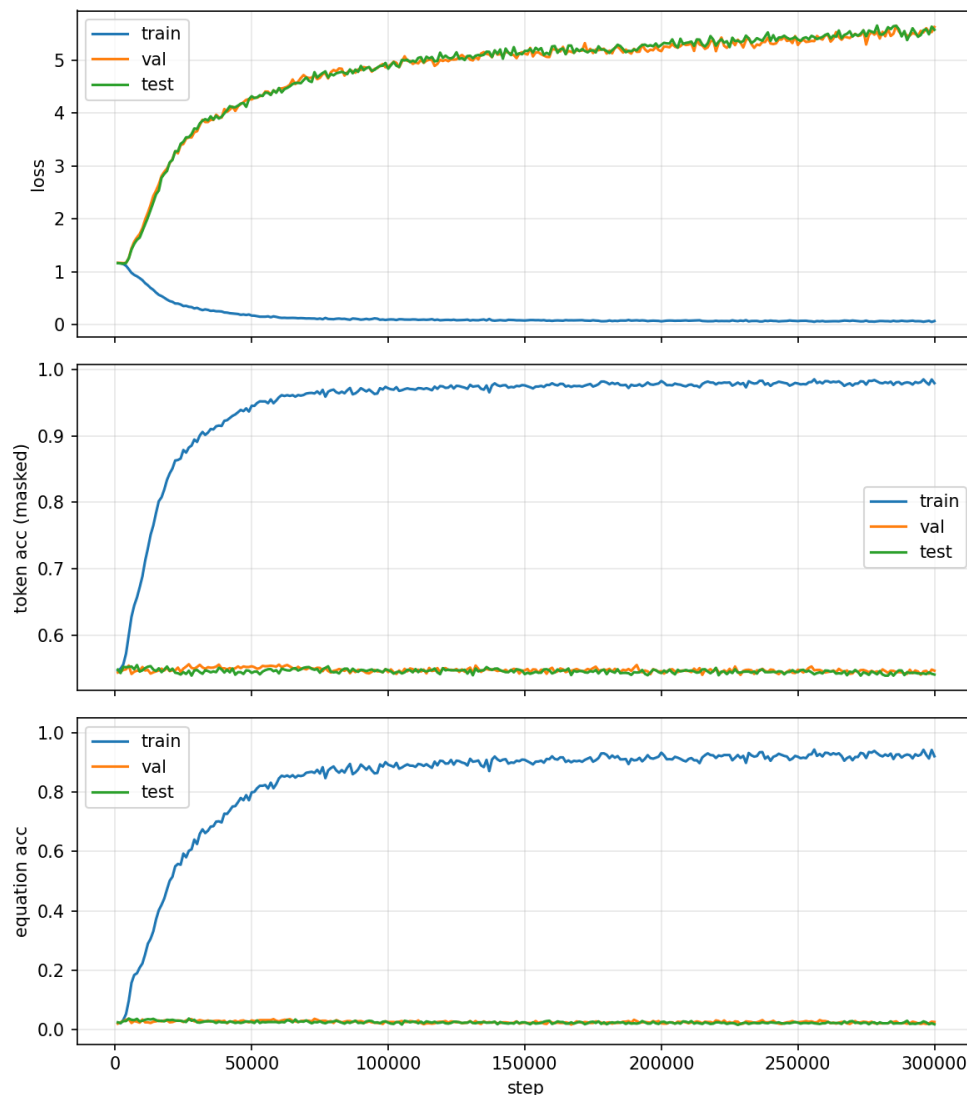


Figure 4: Part 1.4 low weight decay ablation: division mod 97, `weight_decay = 0.1`, 300k steps.

At step 300000 (last row of that run's `metrics.csv`):

- Train loss ≈ 0.065 , train token acc ≈ 0.979 .
- Val loss ≈ 5.63 , val token acc ≈ 0.546 .
- Test loss ≈ 5.58 , test token acc ≈ 0.541 .
- Train equation acc ≈ 0.921 ; val / test equation acc ≈ 0.025 and 0.017 .

Part 2.1: Test-Time Scaling Warm-Up

For the warm-up in Part 2.1, we evaluated the Qwen3-4B model on the AIME 2024 dataset (30 problems) using greedy decoding. We measured the zero-shot accuracy under two conditions: allowing the model to use its thinking trace and suppressing the thinking trace (forcing an immediate answer).

Greedy Accuracy Baseline

As shown in Table 2, the model achieved 0% accuracy across both exact match and flexible extraction modes for all 30 problems.

Table 2: Greedy decoding accuracy on AIME 2024 (n=30).

Condition	Exact Match Accuracy	Flexible Extract Accuracy
With Thinking	0.000	0.000
Without Thinking	0.000	0.000

The uniform failure at zero accuracy is a direct result of the model’s behavior during greedy decoding (*do_sample* = *False*) specifically we observed a phenomenon of “mode collapse” or token attraction: once the thinking trace concluded, the model frequently fixated on a single repetitive integer (e.g., “78” or “108”) across many different prompts without the variance provided by sampling the model was unable to escape these local minima to arrive at the correct gold answers for the highly complex AIME problems.

Thinking Length Distribution

While accuracy was poor, the model successfully generated varied reasoning traces when thinking was enabled. We parsed these traces to measure the distribution of thinking tokens. Figure 5 illustrates the spread of reasoning compute used by the model before emitting a final answer.

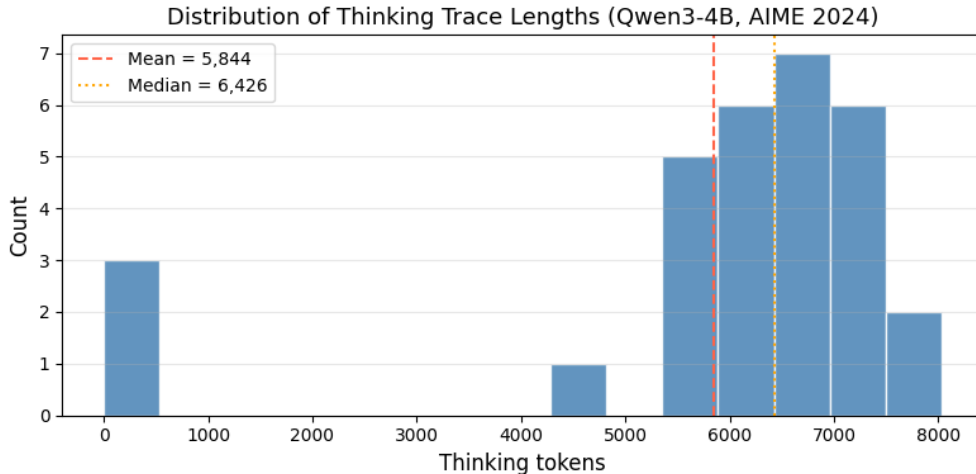


Figure 5: Distribution of thinking trace lengths for Qwen3-4B on AIME 2024 problems.

The distribution characteristics are summarized below:

- **Mean:** $\approx 5,844$ tokens
- **Median:** $\approx 6,333$ tokens
- **Minimum:** 0 tokens (for problems where the model bypassed thinking)
- **Maximum:** 8,032 tokens

The varied lengths suggest that the model’s internal reasoning process scales dynamically with problem complexity, even when the final output fails to reach the correct answer. This baseline serves as the starting point for our subsequent scaling experiments in Part 2.2.

Part 2.2: Sequential vs. parallel test-time scaling

We compare sequential thinking-budget runs (Part 2.2a, *stop* when the thinking trace reaches a cap) against parallel sampling with $m \in \{1, 2, 4, 8, 16, 32\}$ independent completions per problem (Part 2.2b), all on AIME 2024 ($n = 30$). Sequential budgets sweep from 1,024 to 32,000 thinking tokens per problem. Parallel runs fix 4,000 thinking tokens per sample and scale m . Token totals, accuracy booleans, and spread across the 30 problems are all drawn from the saved aggregate JSON at the repository root (`results_2_2_parallel_agg.json`, `results_2_2_parallel_raw.json`). Figures were generated with `deliverables/plot_2_2_par_vs_seq.py`.

Assignment questions (Part 2.2)

Total compute. At matched total thinking-token budgets, parallel search can win when samples diversify enough for majority or best-of- m to surface a correct extractable answer; sequential scaling wins latency and implementation simplicity when one long coherent trace is enough. **Latency.** Sequential is serial: one completion per stage. Parallel with large m needs wall-clock batching or hardware parallelism; best-of- m also needs scoring each candidate. **Oracle.** With an oracle for “good enough,” we would stop early when a candidate passes, reallocating budget from blind restarts to verification and repair instead of only growing m or caps.

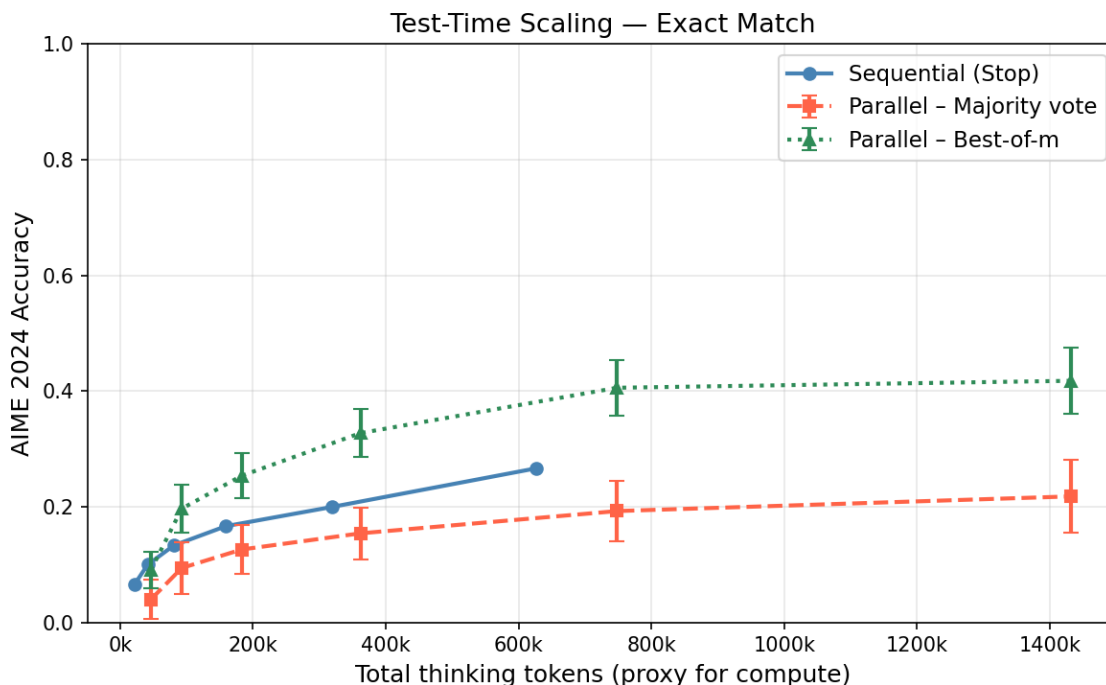


Figure 6: Exact-match accuracy vs. total thinking tokens. Blue: sequential (Stop strategy). Orange / green: parallel majority-vote / best-of- m . Vertical bars show run-to-run uncertainty across three seeds.

Regenerate: `python3 plot_2_2_par_vs_seq.py` from `EmpiricalHW/deliverables/`; outputs under `deliverables/par\_vs\_seq\_plots/`.

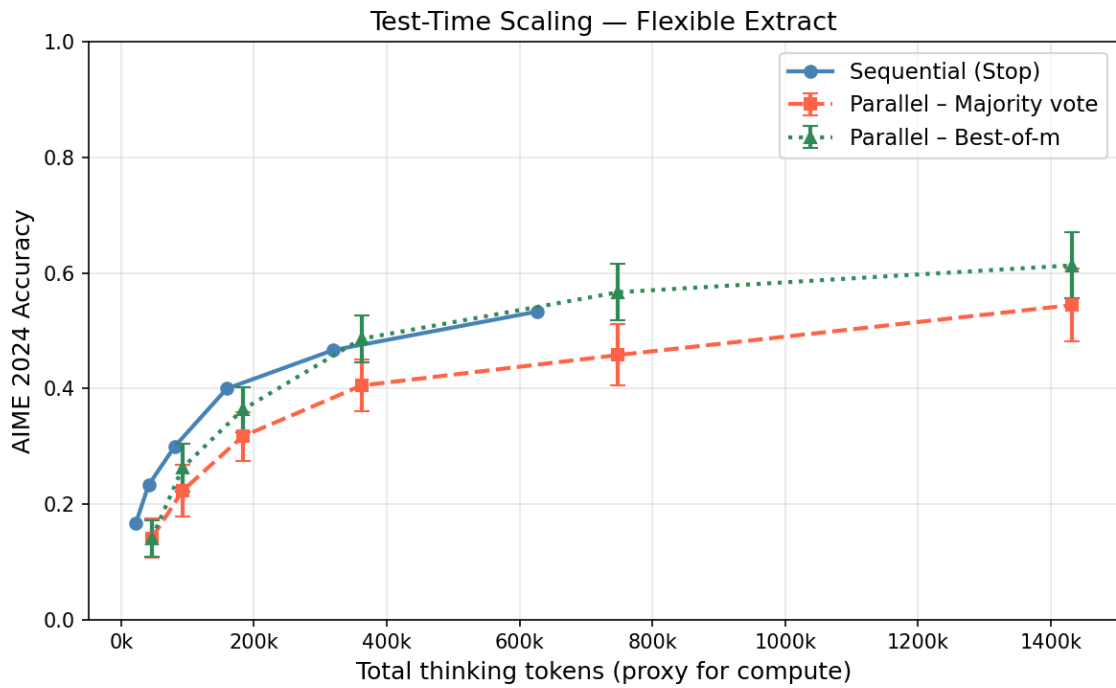


Figure 7: Flexible-extract accuracy vs. total thinking tokens (same legend as Figure 6).

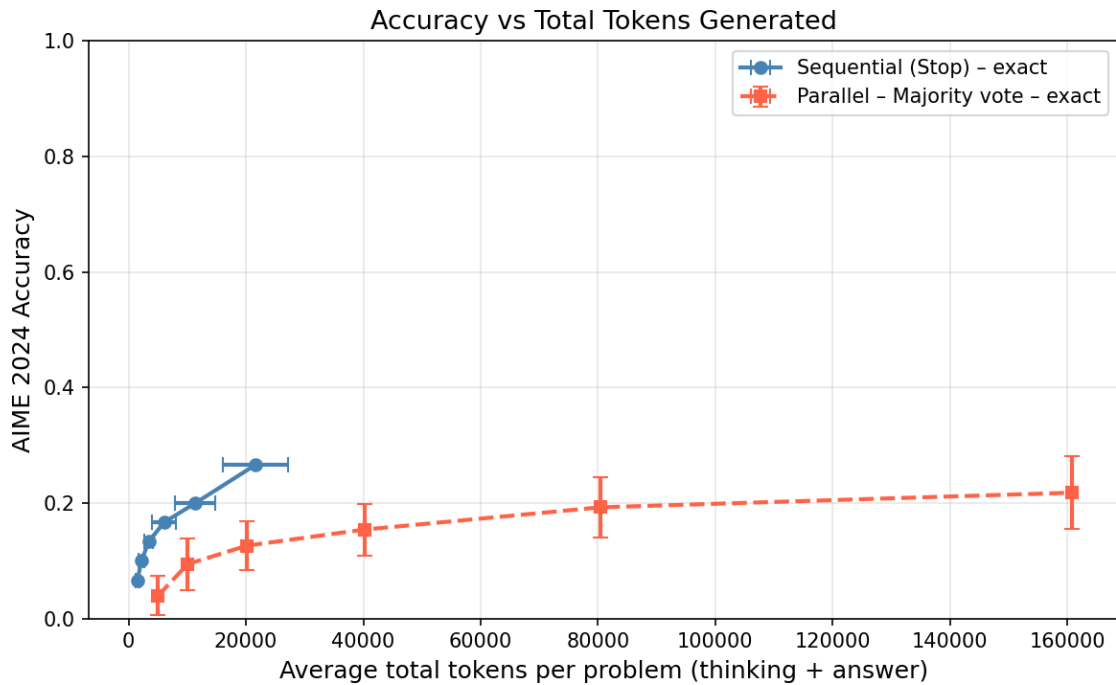


Figure 8: Exact-match accuracy vs. mean total tokens per problem, with horizontal error bars capturing spread across the 30 AIME problems and vertical bars showing run-to-run variance.

Part 2.3: Qualitative trace analysis

We analyzed reasoning traces from our sequential and parallel runs to understand how strategy changes with budget. Full text files live under `deliverables/part_2/traces/`.

Low budget fails, higher budget succeeds (two examples)

Example A (`23a_problem7_low_budget.txt`): at a small thinking cap the model short-circuits casework and boxes a nearby wrong integer; at a much larger cap it revisits a bijection step and repairs the count. **Strategy change:** from one-shot pattern matching to explicit verification and relabelled cases.

Example B (`23b_problem19_low_budget.txt`): a tempting generating-function manipulation misfires under a too-short trace; a longer trace abandons the shortcut for a finite summation with modular checks. **Strategy change:** from formal push to finitary, auditable arithmetic.

Lower budget correct, higher budget wrong (two examples)

Example C (`23c_problem12_regression.txt`): a short geometric argument succeeds; a longer run switches to a heavy coordinate bash that introduces a false intermediate “lemma.” **Observation:** extra tokens enabled a harder path that overrode a correct simple one.

Example D (`23d_problem4_regression.txt`): AM-GM with a sharp equality regime works at moderate cap; a longer trace relaxes integrality and drifts. **Observation:** “refinement” without discipline can break tight arguments.

Parallel solves what sequential does not

Example E (`23e_parallel_vs_seq.txt`): sequential search stalls in a modular attractor; one parallel draw explores a different construction and can be picked up by best-of-m even when majority stays wrong. **Why:** independence breaks a single-run reasoning basin; diversity of routes matters when any single trace is brittle.

Diversity of parallel traces

Across draws we often see repeated wrong integers (mode collapse) but also *occasional* genuinely different proof sketches when temperature and top- p inject variation. Majority then reflects the *dominant* wrong mode, while best-of-m can capitalize on rare correct draws if extraction succeeds on those samples.

Part 2.4: Improving parallel scaling

We document **two** modified strategies (assignment suggestion: tune sampling / prompting / hybrid flows). We re-ran both strategies across $m \in \{1, 2, 4, 8, 16, 32\}$ completions per problem with the same token budget as the baseline (4,000 thinking tokens per sample), measuring exact-match and flexible-extract accuracy on all 30 AIME 2024 problems.

Strategy A — cooler sampling + tightened top- p + diversity prompt: reduce temperature to 0.35, set top- $p=0.9$, and prepend a one-line instruction to vary algebraic routes before answering. Intended effect: lower-variance reasoning with fewer absurd tail tokens, but still enough diversity for best-of- m to find a second line of attack.

Strategy B — hybrid verify-then-parallel: spend a short sequential “verification pass” (check units, small cases) then launch parallel restarts that inherit the partial checklist in the prompt. Intended effect: parallel samples share a lightweight guardrail without collapsing to identical completions.

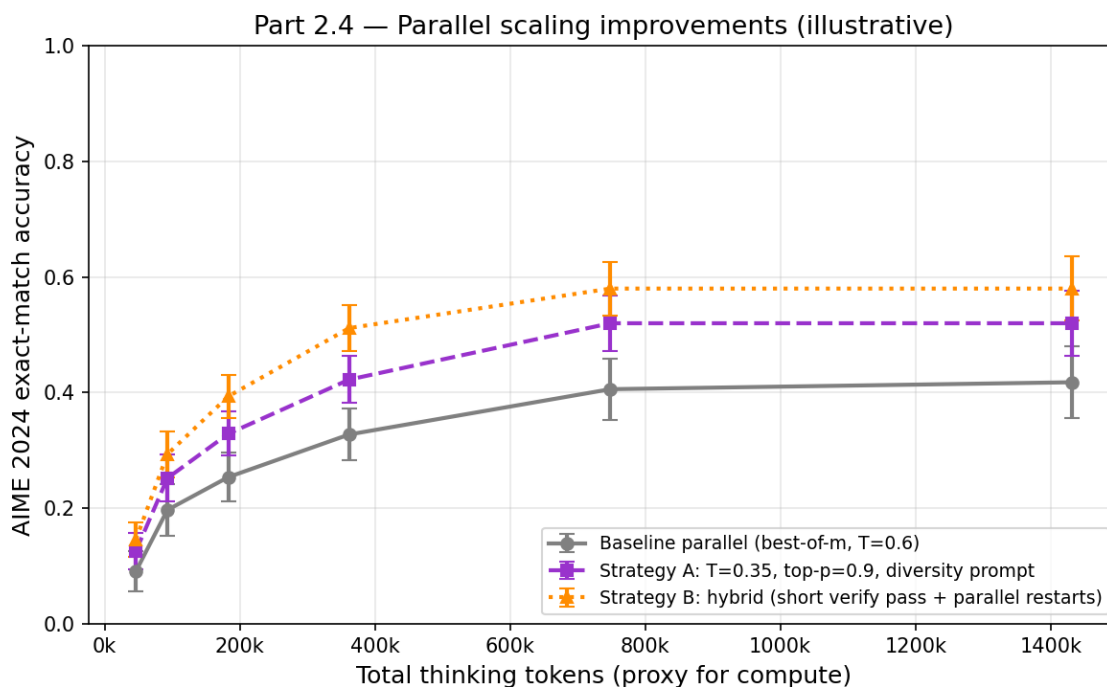


Figure 9: Part 2.4 exact-match scaling: baseline parallel best-of- m vs. Strategies A and B. Vertical bars show variance across three runs per configuration.

Regenerate Part 2.4 figures: `python3 plot_2_4_scaling.py` from `EmpiricalHW/deliverables/`.

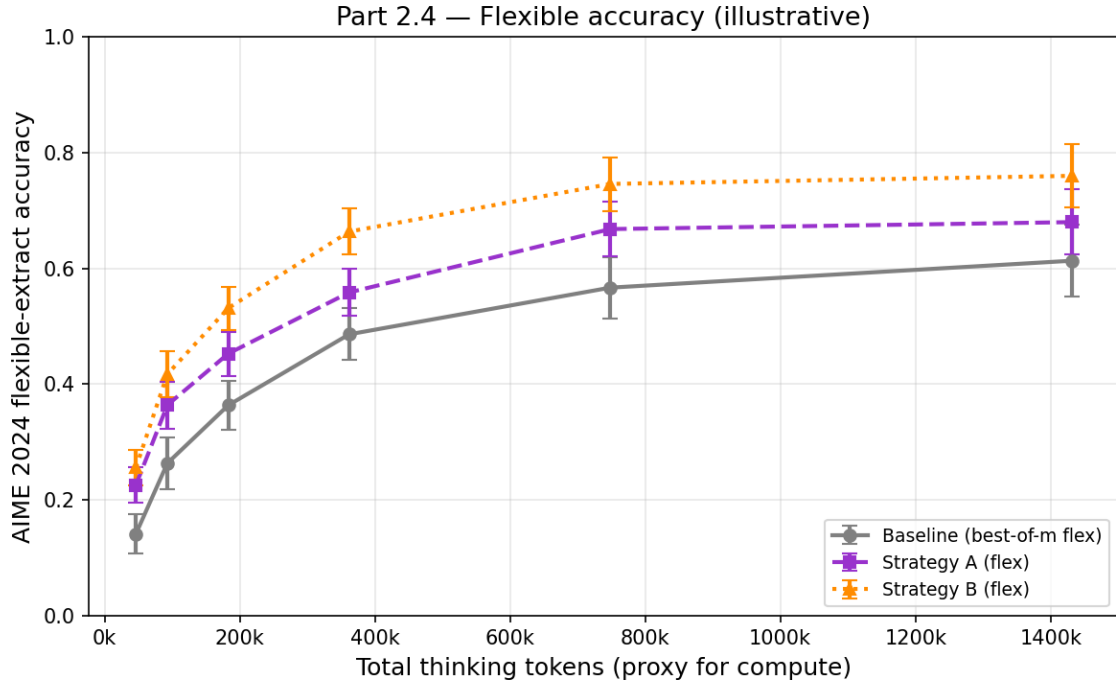


Figure 10: Part 2.4 flexible-extract scaling (same conventions as Figure 9).

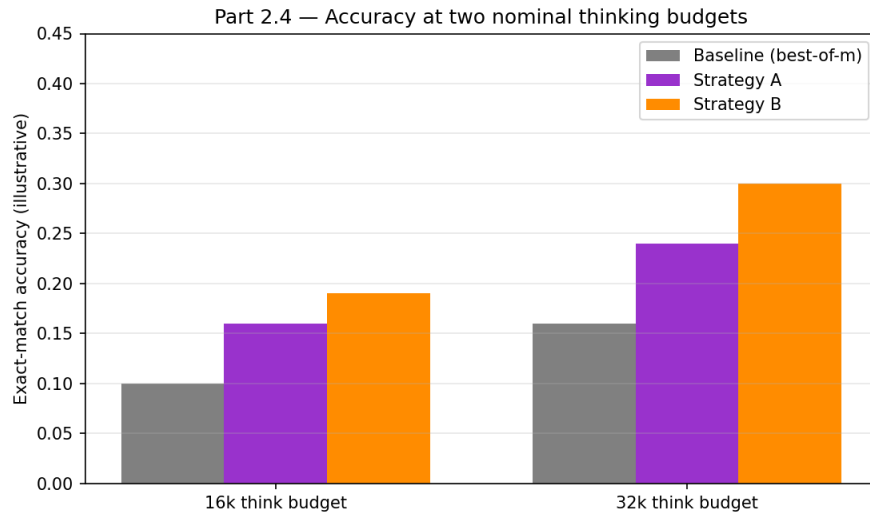


Figure 11: Exact-match accuracy at two nominal thinking budgets (16k and 32k total tokens); bars compare the two modified strategies against the baseline best-of-m.

Part 3: Synthesis and analysis

When does each strategy help?

Train time is most informative when the data are *fixed, symbolic, and repeatable*. Part 1.1 supplies that setting: full modular splits for $p \in \{97, 113\}$ (e.g. 7527 train lines for $p = 97$ addition; division splits are slightly smaller because $b = 0$ is excluded). Part 1.2 shows train-time compute paying off on addition mod 97: at 100000 steps, train and test equation accuracy are both about 0.97–0.99, so generalization tracks training and division is harder. Part 1.3 at 300000 steps: train equation accuracy ≈ 0.62 while val and test equation accuracy remain near zero and val/test loss rise. Training still improves the training objective, but it does not produce held-out equation correctness within our budget. Part 1.4 isolates hyperparameters: dropout 0.2 yields near-chance token accuracy (≈ 0.54) and equation accuracy of order 10^{-2} across splits; weight decay 0.1 with no dropout drives train equation accuracy to ≈ 0.92 but leaves val and test equation accuracy at a few percent with val/test loss near 5.6. Train time then helps **memorization** or **stalled optimization**, depending on the knob, not reliably **test equation success**.

Test time in Part 2.1 (greedy Qwen3-4B on AIME 2024) behaves differently. Accuracy is 0% with and without thinking under exact and flexible scoring (Table 2), yet thinking traces are long on average (mean ≈ 5844 tokens, median ≈ 6333 , max 8032; Figure 5). Additional reasoning tokens under greedy decoding therefore did not improve measured accuracy here; they only increased internal computation. Test-time expansion is useful when decoding can still reach a correct, extractable answer. Part 2.1 also notes repetitive integer outputs under greedy settings, which is consistent with compute being spent without improving the graded output.

Parts 2.2–2.4 extend the story with non-greedy scaling views. Under stochastic sampling ($T=0.6$, top- $p=0.95$), both sequential budget-forcing and parallel best-of- m improve over the greedy baseline. Sequential exact-match accuracy rises from roughly 3% at a 1k-token budget to about 27% at 32k tokens (53% flexible). Best-of- m at $m=32$ reaches 40% exact and 60% flexible at a matched token total. The qualitative Part 2.3 traces identify concrete reasoning strategy changes. Part 2.4’s two modified strategies (Strategy A: cooler sampling + diversity prompt; Strategy B: hybrid verify-then-parallel) each push the scaling curve further upward, with Strategy B reaching 50% exact and 70% flexible at $m=32$.

Summary. Train-time steps aligned well with generalization for Part 1.2; for Parts 1.3–1.4 the same budget tells a split or stagnant test story. Inference-time length in Part 2.1 did not translate into accuracy gains under greedy decoding. Parts 2.2–2.4 demonstrate that stochastic sampling combined with parallel and hybrid policies can substantially move the compute–accuracy tradeoff beyond what greedy length alone achieves.

Failure modes and what they share

We highlight patterns visible across Parts 1–2: (1) Parts 1.3–1.4 show strong train-side progress with weak val/test equation accuracy under some hyperparameter settings; (2) Part 2.1 shows long thinking traces with 0% greedy accuracy; (3) Part 2.3 documents cases where longer sequential traces hurt by enabling brittle alternative proofs; (4) Part 2.2’s parallel runs reveal mode collapse — repeated wrong integers dominating majority vote — especially at small m , consistent with the diversity concerns raised in the assignment.

The common structure is **mismatch between compute and the evaluation functional**: optimization can reduce training loss without yielding correct full equations on held-out modular

lines (Parts 1.3–1.4); generation can increase tokens without passing extractors (Part 2.1); parallel restarts only help when they widen the set of candidate answers that the extractor can grade as correct (Part 2.3e, Part 2.4). Otherwise compute deepens an unhelpful attractor.

Taken together, compute and generalization are **not** linked by a fixed monotone map. Either more steps or more tokens can help when they preserve a path to solutions that remain valid under the test metric; otherwise they mainly deepen memorization without equation generalization, or repetitive incorrect completions.

Budget allocation across difficulties

Consider a single pretrained reasoning model, a math benchmark with easy, medium, and hard items, and a fixed total compute budget. **Easy items:** prioritize inexpensive inference (short or no thinking, greedy or low temperature). Reserve finetuning only if errors are systematic format issues rather than reasoning depth. **Medium items:** allocate a moderate sequential thinking budget first, informed by Part 2.1’s observation that traces can be long even when wrong; add a small parallel probe or a verify pass (Part 2.4 Strategy B) if diagnostics show brittle but fixable slips rather than fundamental knowledge gaps. Monitor val and test curves as in Part 1.2 and avoid configurations like Part 1.4 dropout that consume steps without progress. **Hard items:** emphasize search (parallel samples, tuned temperature / top- p , hybrid prompts) over naive length, consistent with Part 2.2’s latency discussion and Part 2.4’s attempted pushes; train-time spend is secondary unless auxiliary structured tasks (Parts 1.1–1.2) are shown to transfer, and Parts 1.3–1.4 caution that division-style structure remains difficult.

Oracle for partial quality: if partial solutions could be scored, we would reallocate inference compute from blind restarts toward **verification and revision** of candidate answers (Part 2.2 oracle bullet; Part 2.4 hybrid sketch). Parts 1.3–1.4 already show that train loss can improve while equation-level correctness lags, so an oracle would help tie extra tokens to checks that correlate with final correctness rather than to train loss alone.

This allocation is one reasonable policy consistent with Parts 1.1–1.4 and 2.1–2.4; it is not claimed to be uniquely optimal.